Unidad 02
Programación Multihilo



Módulo Programación de Servicios y Procesos

modalidad @vanza

UD02 – Programación Multihilo

1.- Introducción

La **programación multihilo** o multithrearing consiste en desarrollar programas o aplicaciones de flujo múltiple. Cada uno de esos flujos de ejecución es un thread o **hilo**.

2.- Conceptos sobre hilos

- Un hilo no puede existir independientemente de un proceso.
- Un hilo no puede ejecutarse por si solo.
- Dentro de cada proceso puede haber varios hilos ejecutándose.

2.1.- Recursos compartidos por los hilos

2.2.- Ventajas y uso de hilos

UD02 – Programación Multihilo

3.- Multihilos en java. Librerías y clases:

3.1.- Utilidades de concurrencia del paquete java.lang.

Clase thread, Interfaz Runnable

3.2.- Utilidades de concurrencia del paquete java.util.concurrent.

Clases de sincronización. Semaphore, CountDownLatch, CyclicBarrier y Exchanger.

Interfaces para separar la lógica de la ejecución: Executor, Callable y Future.

4.- Creación de hilos.

se pueden **implementar o definir de dos formas**:

- Extendiendo la clase thread.
- Mediante la interfaz Runnable .

Ejemplo 

```
public class Main {
    public static void main(String[] args) {
    System.out.println("¡Hola mundo!\n");
    Thread miHilo = Thread.currentThread();
    System.out.println("Por defecto, el hilo que ejecuta el método main() "
            +"de mi programa se llama '" + miHilo.getName() + "'\n");
  }
}
```

4.- Creación de hilos.

4.1.- Hilos extendidos.

Ejemplo




```
public class Saludo extends Thread {
//clase  que extiende a Thread
   public void run() {
   // se redefine el método run() con el código asociado al hilo
        System.out.println(";Saludo desde un hilo extendiendo thread!");
   }
   public static void main(String args[]) {
        Saludo hilo1=new Saludo();
        //se crea un objeto Thread, el hilo hilo1
        hilo1.start();
        //invoca a start() y pone en marcha el hilo hilo1
   }
 }
```

UD02 – Programación Multihilo

4.- Creación de hilos.

4.2.- Creación de hilos mediante la interfaz Runnable.

Ejemplo




```
public class Saludo implements Runnable {
//clase que implementa a Runnable
    public void run() {
    //se redefine el método run()con el código asociado al hilo
        System.out.println("¡Saludo desde un hilo creado con Runnable!");
    }
    public static void main(String args[]) {
       Saludo  miRunnable=new Saludo();
       //se crea un objeto  Saludo
       Thread hilo1= new Thread(miRunnable);
       //se crea un objeto Thread (el hilo hilo1) pasando como argumento
      // al constructor un objeto Saludo
        hilo1.start();
       //se invoca al método start() del hilo hilo1
    }
}
```

UD02 – Programación Multihilo

5.- Estados de un hilo.

5.1.- Iniciar `start();`

5.2.- Detener `sleep(); wait();`

5.3.- Finalizar `isAlive();`

5.4.- Ejemplo. Dormir

Ejemplo 

Ejemplo 

UD02 – Programación Multihilo

6.- Gestion y planificación de hilos.

6.1.- Prioridad

`MAX_PRIORITY, NORM_PRIORITY, MIN_PRIORITY`

6.2.- Egosita y expulsora

Ejemplo 

UD02 – Programación Multihilo

7.- Sincronización y comunicación de hilos.

7.1.- Información compartida

7.2.- Monitores

7.3.- Synchronized

7.4.- Comunicación entre hilos.

Ejemplos 

7.5.- Interbloqueo: uno a más hilos se bloquean

7.6.- Semaphore: permite definir un semáforo para controlar el acceso a un recurso compartido

7.7.- Exchanger: un punto de sincronización donde se intercambian objetos entre dos hilos

7.8.- CountDownLatch: uno o más threads esperen hasta que otros threads finalicen su trabajo.

7.9.- CyclicBarrier: uno o más threads se esperen hasta que todos ellos finalicen su trabajo.

UD02 – Programación Multihilo

8.- Aplicaciones multihilo.

8.1.- Otras utilidades de concurrencia

8.2.- Interfaz executor y los pools

8.3.- Gestión de excepciones

8.4.- Depuración y documentación

Ejemplos 